

ARCore Cloud Anchors (含永久 Cloud Anchors)

來源：<https://codelabs.developers.google.com/codelabs/arcore-cloud-anchors?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將瞭解如何使用 ARCore Cloud Anchors 服務，在多部裝置上建立共同參照影格 (具有相同的位置和方向)，運用 Cloud Anchor 打造共用 AR 體驗。

目錄

1. [總覽](#)
2. [設定開發環境](#)
3. [主持 Anchor 節目](#)
4. [儲存 ID 和解析錨點](#)
5. [在裝置之間分享](#)
6. [總結](#)

步驟 1 · 約 0 分鐘

1. 總覽

ARCore 平台可讓您在行動裝置上建構擴增實境應用程式。透過 Cloud Anchors API，您可以建立共用參照架構的 AR 應用程式，讓多位使用者在同一處現實世界位置放置虛擬內容。

這個程式碼研究室會逐步引導您瞭解 Cloud Anchors API。您將使用現有的 ARCore 應用程式，修改為使用 Cloud Anchors，並建立共用的 AR 體驗。

ARCore 錨點和持續性雲端錨點

ARCore 的基本概念是**錨點**，用來描述現實世界中的固定位置。ARCore 會隨著時間提升動作追蹤功能，並自動調整錨點的姿勢值。

雲端錨點是代管在雲端的錨點。多位使用者可以解決這些問題，在使用者和裝置之間建立共同的參照架構。



代管錨點

當錨點代管時，會發生以下情況：

1. 系統會將錨點相對於世界的姿態上傳至雲端，並取得 Cloud Anchor ID。
Cloud Anchor ID 是字串，必須傳送給想解析這個錨點的任何人。
2. 含有錨點視覺資料的資料集會上傳至 Google 伺服器。
這個資料集包含裝置最近看到的視覺資料。在代管前，稍微移動裝置，從不同角度擷取錨點周圍的區域，可獲得更準確的定位結果。

轉移 Cloud Anchor ID

在本程式碼研究室中，您將使用 **Firebase** 轉移 Cloud Anchor ID。您可以使用其他方式分享 Cloud Anchor ID。

解析錨點

您可以使用 Cloud Anchor API，透過雲端錨點 ID 解析錨點。這會在與原始代管錨點相同的實體位置建立新錨點。解決問題時，裝置必須查看與原始託管錨點相同的實體環境。

永久雲端錨點

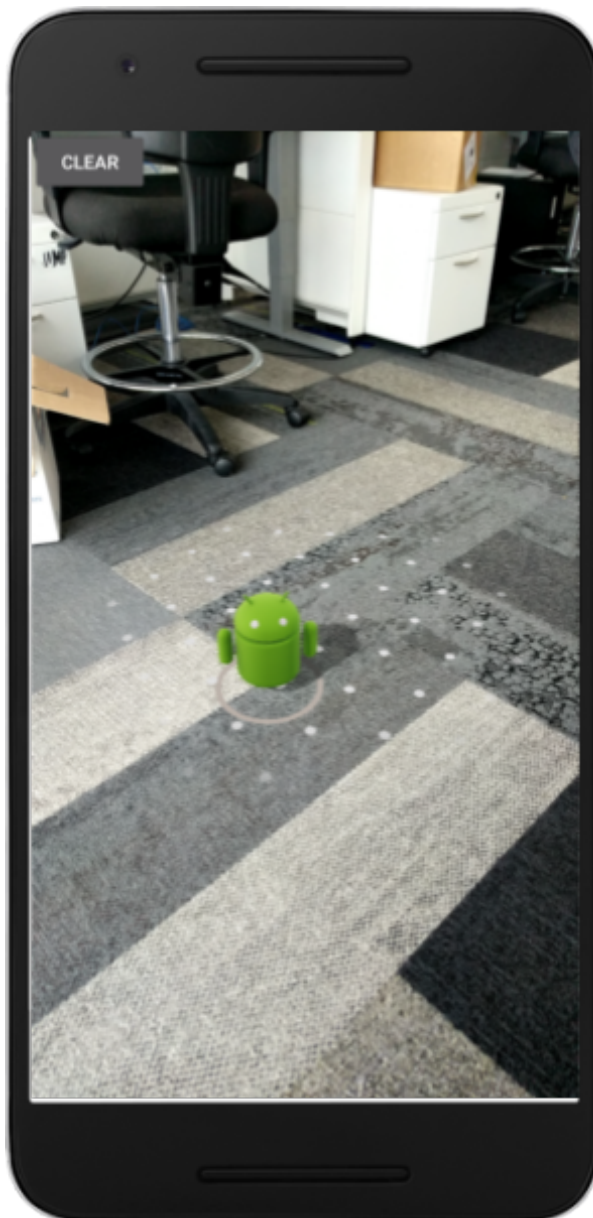
在 1.20 版之前，雲端錨點只能在代管後 24 小時內解析。使用 Persistent Cloud Anchors API，您可以建立雲端錨點，並在建立後 1 天到 365 天內解析。

建構目標

在本程式碼研究室中，您將以現有的 ARCore 應用程式為基礎進行建構。完成本程式碼研究室後，您的應用程式將具備下列功能：

- 能夠託管永久性雲端錨點，並取得雲端錨點 ID。
- 在裝置上儲存 Cloud Anchor ID，方便使用 Android `SharedPreferences` 輕鬆擷取。
- 使用已儲存的 Cloud Anchor ID，解析先前代管的錨點。這樣一來，我們就能輕鬆使用單一裝置模擬多使用者體驗，以進行本程式碼研究室。
- 與執行相同應用程式的其他裝置共用 Cloud Anchor ID，讓多位使用者在相同位置看到 Android 雕像。

在 Cloud Anchor 的位置算繪 Android 雕像：



課程內容

- 瞭解如何使用 ARCore SDK 代管錨點，並取得 Cloud Anchor ID。
- 如何使用 Cloud Anchor ID 解析錨點。
- 如何在同一部裝置或不同裝置上，儲存及共用不同 AR 工作階段之間的 Cloud Anchor ID。

軟硬體需求

- [支援 ARCore 的裝置](#)，透過 USB 傳輸線連接至開發機器。
 - [Google Play 服務 - AR 適用](#) 1.22 以上版本。
 - 搭載 [Android Studio](#) (3.0 以上版本) 的開發電腦。
-

步驟 2 · 約 0 分鐘

2. 設定開發環境

設定開發機器

使用 USB 傳輸線將 ARCore 裝置連接至電腦。確認裝置[允許 USB 偵錯](#)。

開啟終端機並執行 `adb devices`，如下所示：

```
adb devices

List of devices attached
<DEVICE_SERIAL_NUMBER>    device
```

`<DEVICE_SERIAL_NUMBER>` 是裝置專屬的字串。請先確認只有一部裝置，再繼續操作。

下載及安裝程式碼

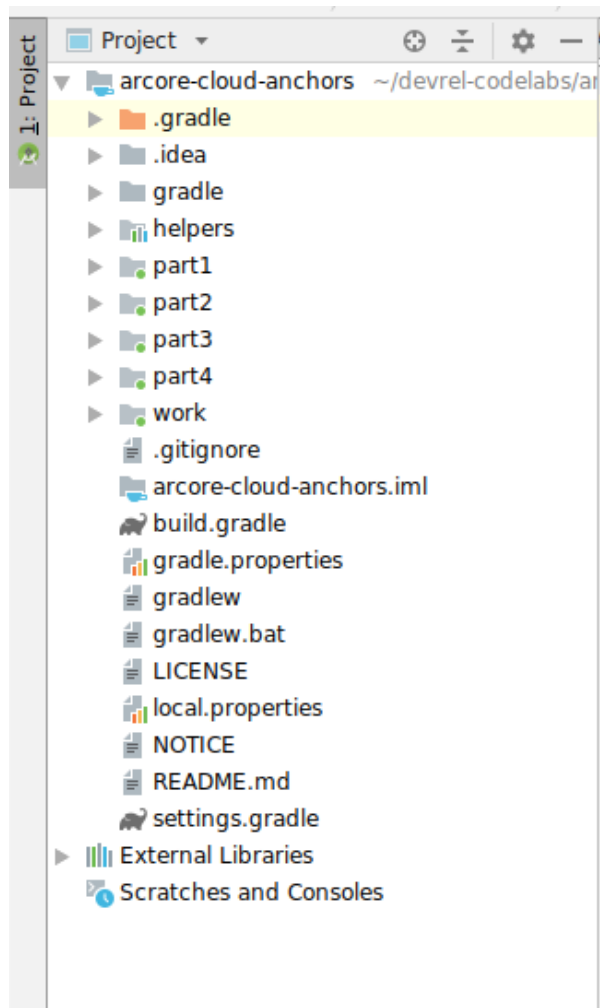
您可以複製存放區：

```
git clone https://github.com/googlecodelabs/arcore-cloud-anchors.git
```

或者下載並解壓縮 ZIP 檔案：

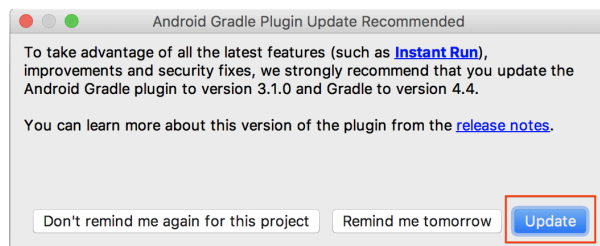
請啟動 Android Studio。按一下「Open an existing Android Studio project」。然後前往您解壓縮上述下載的 ZIP 檔案的目錄，並按兩下 `arcore-cloud-anchors` 目錄。

這是含有多個模組的單一 Gradle 專案。如果 Android Studio 左上方的「Project」窗格尚未顯示，請按一下下拉式選單中的「Projects」。結果應如下所示：



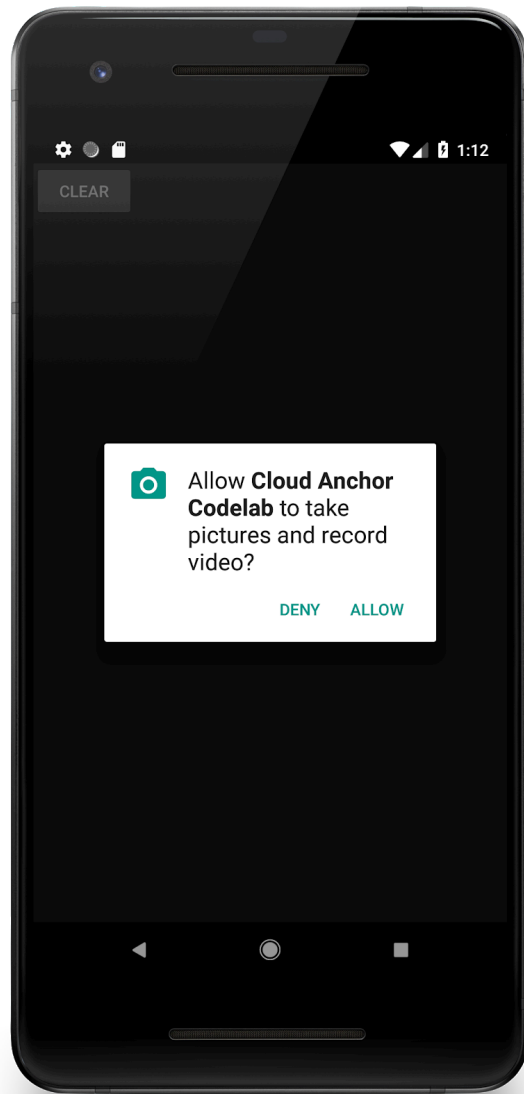
您主要會在 `work` 模組中作業。其他模組包括 `helpers` 模組，其中包含您會使用的一組實用包裝函式類別。此外，本程式碼研究室的每個部分都有完整解決方案。除了 `helpers` 模組外，每個模組都是可建構的應用程式。

如果看到建議升級 Android Gradle 外掛程式的對話方塊，請按一下「Don't remind me again for this project」：



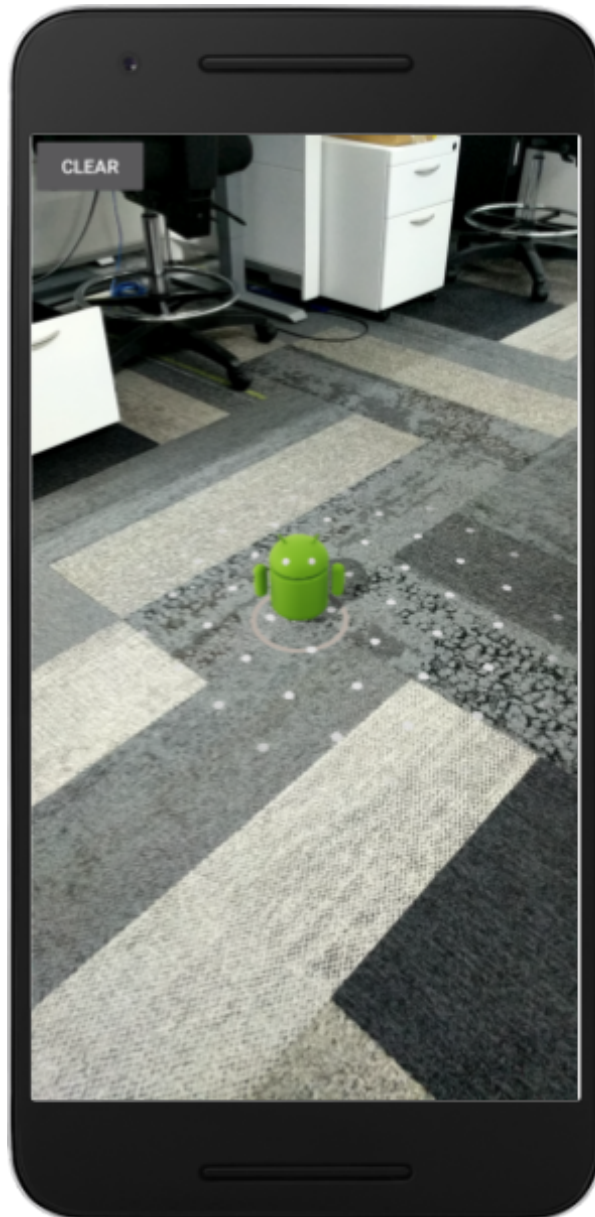
依序按一下「Run」(執行) > 「Run...」(執行...) > 'work'。在顯示的「Select Deployment Target」對話方塊中，裝置應會列在「Connected Devices」下方。選取裝置，然後按一下「OK」。Android Studio 會建構初始應用程式，並在裝置上執行。

首次執行應用程式時，系統會要求 `CAMERA` 權限。輕觸「允許」繼續操作。



如何使用應用程式

1. 移動裝置，協助應用程式尋找飛機。找到平面時，系統會以虛線顯示平面。
2. 輕觸平面上的某處來放置錨點。系統會在錨點位置繪製 Android 機器人。這個應用程式一次只能放置一個錨點。
3. 移動裝置。即使裝置移動，人偶也應停留在相同位置。
4. 按一下「清除」按鈕即可移除錨點。這樣就能放置另一個錨點。



這與 ARCore SDK 發布的 [Hello AR 範例](#) 類似。

目前這個應用程式只會使用 ARCore 提供的動作追蹤功能，在單次執行應用程式時追蹤錨點。如果您決定結束、終止並重新啟動應用程式，先前放置的錨點和所有相關資訊 (包括姿勢) 都會遺失。

在接下來的幾節中，您將以這個應用程式為基礎，瞭解如何在 AR 工作階段之間共用錨點。

3. 主持 Anchor 節目

在本節中，您將修改 `work` 專案來代管錨點。編寫程式碼前，您需要對應用程式的設定進行幾項修改。

宣告 INTERNET 權限

由於雲端錨點需要與 ARCore Cloud Anchor API 服務通訊，因此應用程式必須具備網際網路存取權。

在 `AndroidManifest.xml` 檔案中，於 `android.permission.CAMERA` 權限宣告下方新增下列程式碼：

```
<!-- Find this line... -->
<uses-permission android:name="android.permission.CAMERA"/>

<!-- Add the line right below -->
<uses-permission android:name="android.permission.INTERNET"/>
```

啟用 ARCore API

1. 前往 [ARCore API](#) 服務頁面。
2. 在專案清單中選取所需專案，或是建立新的專案。
3. 按一下「啟用」。

設定免密碼驗證

如果要使用永久雲端錨點，您必須使用免金鑰驗證，向 ARCore API 進行驗證。

1. 前往 [Google Cloud Platform Console](#)。
2. 在專案清單中選取一個專案。
3. 如果「API 和服務」頁面尚未開啟，請開啟主控台左側選單，然後選取「API 和服務」。
4. 按一下左側的「憑證」。
5. 按一下「建立憑證」，然後選取「OAuth 用戶端 ID」。
6. 填入下列值：
 - 應用程式類型：Android
 - 套件名稱：`com.google.ar.core.codelab.cloudanchor`
7. 擷取偵錯簽署憑證指紋：
 1. 在 Android Studio 專案中，開啟 **Gradle 工具窗格**。
 2. 在 **cloud-anchors > work > Tasks > android** 中，執行 **signingReport** 工作。
 3. 將 SHA-1 指紋複製到 Google Cloud 的「SHA-1 憑證指紋」欄位。

設定 ARCore

接著，您將修改應用程式，在使用者輕觸時代管錨點，而非一般錨點。如要這麼做，您必須設定 ARCore 階段作業，才能啟用 Cloud Anchors。

在 `CloudAnchorFragment.java` 檔案中，新增下列程式碼：

```
// Find this line...
session = new Session(requireActivity());

// Add these lines right below:
// Configure the session.
Config config = new Config(session);
config.setCloudAnchorMode(CloudAnchorMode.ENABLED);
session.configure(config);
```

請先建構並執行應用程式，再繼續操作。請務必只建構 `work` 模組。應用程式應可順利建構，並照常執行。

主持錨點

現在可以代管要上傳至 ARCore API 的錨點。

在 `CloudAnchorFragment` 類別中新增下列欄位：

```
// Find this line...
private Anchor currentAnchor = null;

// Add these lines right below.
@Nullable
private Future future = null;
```

請記得加入 `com.google.ar.core.Future` 的匯入內容。

按照下列方式修改 `onClearButtonPressed` 方法：

```
private void onClearButtonPressed() {
    // Clear the anchor from the scene.
    if (currentAnchor != null) {
        currentAnchor.detach();
        currentAnchor = null;
    }

    // The next part is the new addition.
    // Cancel any ongoing asynchronous operations.
    if (future != null) {
        future.cancel();
        future = null;
    }
}
```

接著，將下列方法新增至 `CloudAnchorFragment` 類別：

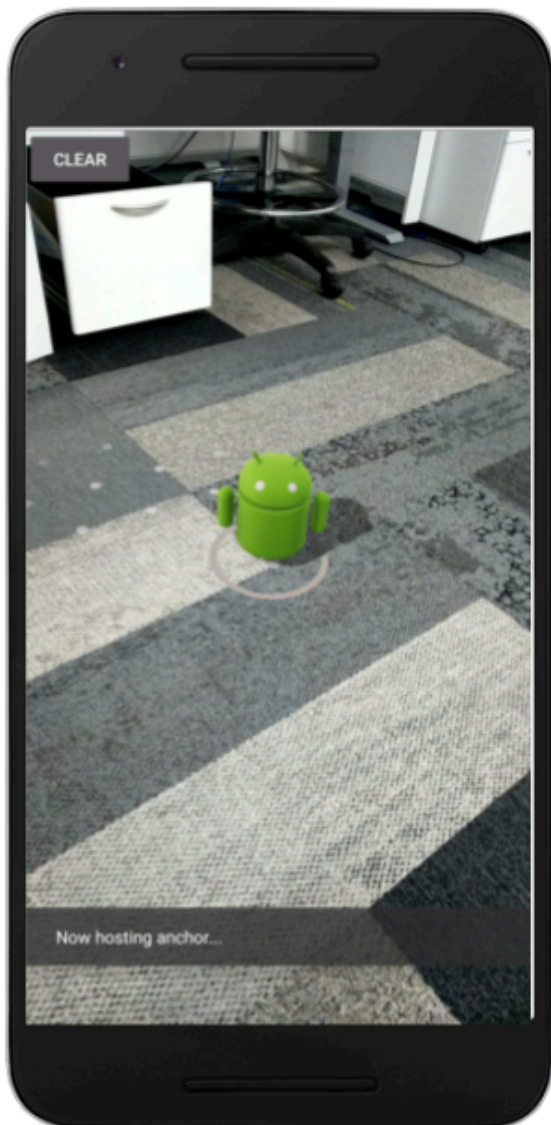
```
private void onHostComplete(String cloudAnchorId, CloudAnchorState cloudState) {
    if (cloudState == CloudAnchorState.SUCCESS) {
        messageSnackBarHelper.showMessage(getActivity(), "Cloud Anchor Hosted. ID: " +
cloudAnchorId);
    } else {
        messageSnackBarHelper.showMessage(getActivity(), "Error while hosting: " +
cloudState.toString());
    }
}
```

在 `CloudAnchorFragment` 類別中找出 `handleTap` 方法，然後新增以下程式碼行：

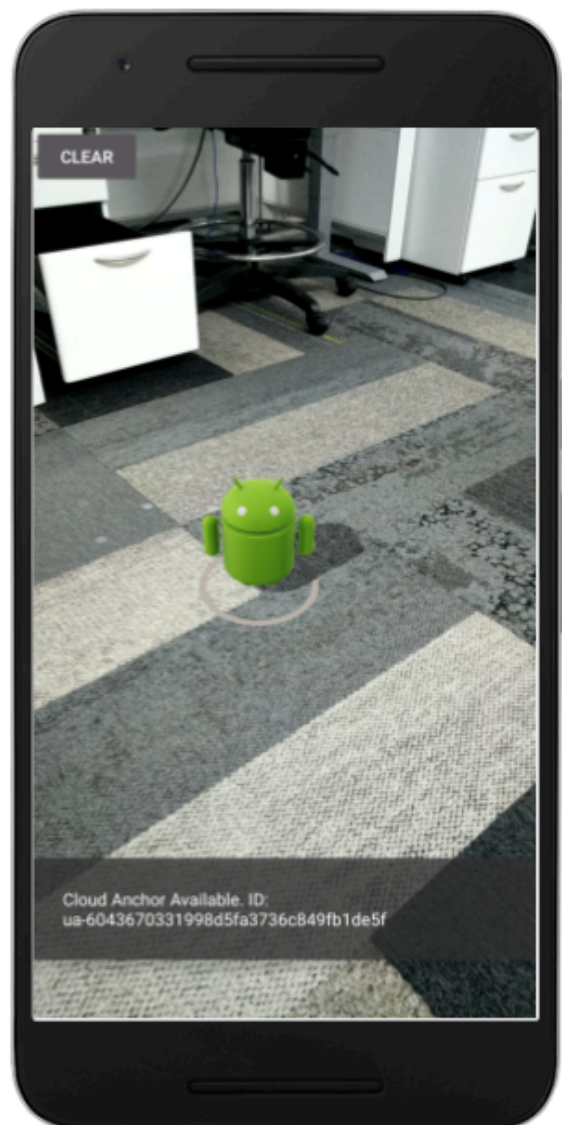
```
// Find this line...
currentAnchor = hit.createAnchor();

// Add these lines right below:
messageSnackBarHelper.showMessage(getActivity(), "Now hosting anchor...");
future = session.hostCloudAnchorAsync(currentAnchor, 300, this::onHostComplete);
```

再次從 Android Studio 執行應用程式。放置錨點時，您應該會看到「**Now hosting anchor...**」訊息。主機代管成功後，您應該會看到另一則訊息。如果看到「**Error hosting anchor: ERROR_NOT_AUTHORIZED**」，請確認 OAuth 用戶端設定正確無誤。



Immediately after placing an anchor



After waiting for a bit

如要進一步瞭解錨點代管期間發生的錯誤，請參閱 [Cloud Anchor 狀態說明文件](#)。

只要知道錨點 ID，且與錨點位於同一實體空間，就能使用錨點 ID，在與周圍環境完全相同的姿態 (位置和方向) 建立錨點。

不過，錨點 ID 很長，其他使用者不容易手動輸入。在接下來的章節中，您將以容易擷取的方式儲存 Cloud Anchor ID，以便在相同或另一部裝置上解析錨點。

步驟 4 · 約 0 分鐘

4. 儲存 ID 和解析錨點

在本節中，您會將短代碼指派給長 Cloud Anchor ID，方便其他使用者手動輸入。您將使用 [Shared Preferences](#) API，將 Cloud Anchor ID 儲存為鍵/值表格中的值。即使應用程式終止並重新啟動，這個資料表仍會保留。

我們已為您提供名為 `StorageManager` 的輔助類別。這是 `SharedPreferences` API 的包裝函式，內含產生新不重複短代碼，以及讀取/寫入 Cloud Anchor ID 的方法。

使用 StorageManager

修改 `CloudAnchorFragment`，使用 `StorageManager` 儲存具有短代碼的 Cloud Anchor ID，方便日後擷取。

在「`CloudAnchorFragment`」中建立下列新欄位：

```
// Find this line...
private TapHelper tapHelper;

// And add the storageManager.
private final StorageManager storageManager = new StorageManager();
```

然後修改 `onHostComplete` 方法：

```
private void onHostComplete(String cloudAnchorId, CloudAnchorState cloudState) {
    if (cloudState == CloudAnchorState.SUCCESS) {
        int shortCode = storageManager.nextShortCode(getActivity());
        storageManager.storeUsingShortCode(getActivity(), shortCode, anchor.getCloudAnchorId());
        messageSnackbarHelper.showMessage(
            getActivity(), "Cloud Anchor Hosted. Short code: " + shortCode);
    } else {
        messageSnackbarHelper.showMessage(getActivity(), "Error while hosting: " +
cloudState.toString());
    }
}
```

現在，請從 Android Studio **建構及執行應用程式**。建立及代管錨點時，系統會顯示短代碼，而非長版的 Cloud Anchor ID。

放置錨點後立即	稍待片刻後
---------	-------



請注意，`StorageManager` 目前一律會依遞增順序指派短代碼。

接著，您會新增幾個 UI 元素，以便輸入短代碼並重新建立錨點。

新增「解決」按鈕

您會在「CLEAR」**CLEAR**按鈕旁新增另一個按鈕。即「解決」按鈕。按一下「解決」按鈕，系統會開啟對話方塊，提示使用者輸入短代碼。短代碼用於從 `StorageManager` 擷取 Cloud Anchor ID，並解析錨點。

如要新增按鈕，請修改 `res/layout/cloud_anchor_fragment.xml` 檔案。在 Android Studio 中按兩下檔案，然後按一下底部的「Text」分頁標籤，即可顯示原始 XML。進行下列修改：

```

<!-- Find this element. -->
<Button
    android:text="CLEAR"
    android:id="@+id/clear_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>

<!-- Add this element right below. -->
<Button
    android:text="RESOLVE"
    android:id="@+id/resolve_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>

```

現在，請在 `CloudAnchorFragment` 中新增欄位：

```
private Button resolveButton;
```

新增方法：

```

private void onResolveButtonPressed() {
    ResolveDialogFragment dialog = new ResolveDialogFragment();
    dialog.show(getFragmentManager().getSupportFragmentManager(), "Resolve");
}

```

在 `onCreateView` 方法中初始化 `resolveButton`，如下所示：

```

// Find these lines...
Button clearButton = rootView.findViewById(R.id.clear_button);
clearButton.setOnClickListener(v -> onClearButtonPressed());

// Add these lines right below.
resolveButton = rootView.findViewById(R.id.resolve_button);
resolveButton.setOnClickListener(v -> onResolveButtonPressed());

```

找出 `handleTap` 方法，並修改為：

```

private void handleTap(Frame frame, Camera camera) {
    // ...

    // Find this line.
    currentAnchor = hit.createAnchor();

    // Add this line right below.
    getActivity().runOnUiThread(() -> resolveButton.setEnabled(false));
}

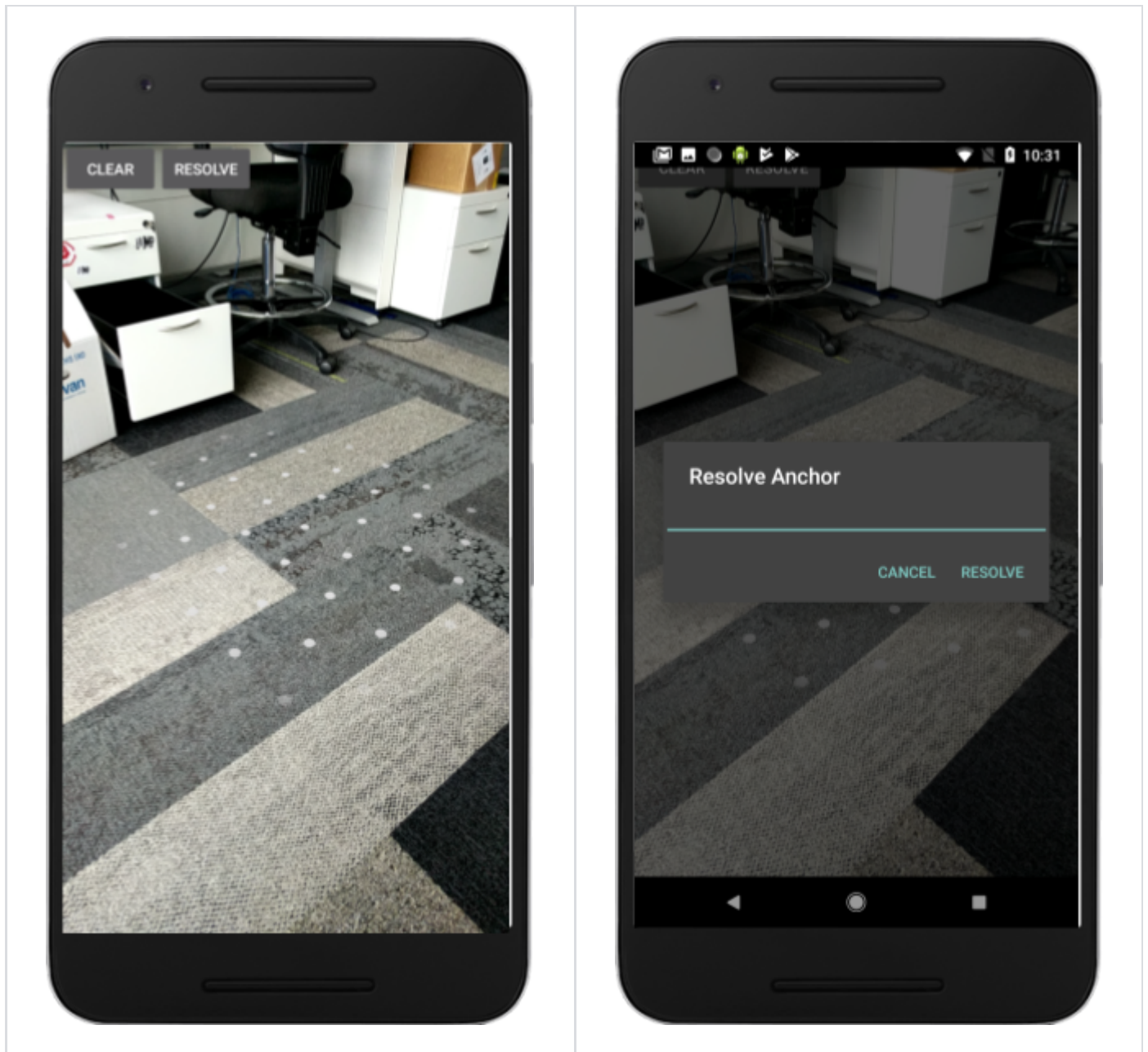
```

在 `onClearButtonPressed` 方法中新增一行：


```
private void onClearButtonPressed() {  
    // Clear the anchor from the scene.  
    if (currentAnchor != null) {  
        currentAnchor.detach();  
        currentAnchor = null;  
    }  
  
    // Cancel any ongoing async operations.  
    if (future != null) {  
        future.cancel();  
        future = null;  
    }  
  
    // The next line is the new addition.  
    resolveButton.setEnabled(true);  
}
```

從 Android Studio **建構及執行應用程式**。「清除」按鈕旁應該會顯示「解決」按鈕。按一下「RESOLVE」按鈕後，應該會彈出如下所示的對話方塊。

「解決」按鈕現在會顯示	點選按鈕後，系統會顯示這個對話方塊
-------------	-------------------



輕觸平面並放置錨點後，「RESOLVE」按鈕應該會停用，但輕觸「CLEAR」按鈕後，應該會再次啟用。這是刻意設計，確保場景中一次只會有一個錨點。

「Resolve Anchor」對話方塊不會執行任何動作，但您現在要變更這點。

解析錨點

在 `CloudAnchorFragment` 類別中新增下列方法：

```

private void onShortCodeEntered(int shortCode) {
    String cloudAnchorId = storageManager.getCloudAnchorId(getActivity(), shortCode);
    if (cloudAnchorId == null || cloudAnchorId.isEmpty()) {
        messageSnackBarHelper.showMessage(
            getActivity(),
            "A Cloud Anchor ID for the short code " + shortCode + " was not found.");
        return;
    }
    resolveButton.setEnabled(false);
    future = session.resolveCloudAnchorAsync(
        cloudAnchorId, (anchor, cloudState) -> onResolveComplete(anchor, cloudState,
shortCode));
}

private void onResolveComplete(Anchor anchor, CloudAnchorState cloudState, int shortCode) {
    if (cloudState == CloudAnchorState.SUCCESS) {
        messageSnackBarHelper.showMessage(getActivity(), "Cloud Anchor Resolved. Short code: " +
shortCode);
        currentAnchor = anchor;
    } else {
        messageSnackBarHelper.showMessage(
            getActivity(),
            "Error while resolving anchor with short code "
                + shortCode
                + ". Error: "
                + cloudState.toString());
        resolveButton.setEnabled(true);
    }
}
}

```

然後修改 `onResolveButtonPressed` 方法：

```

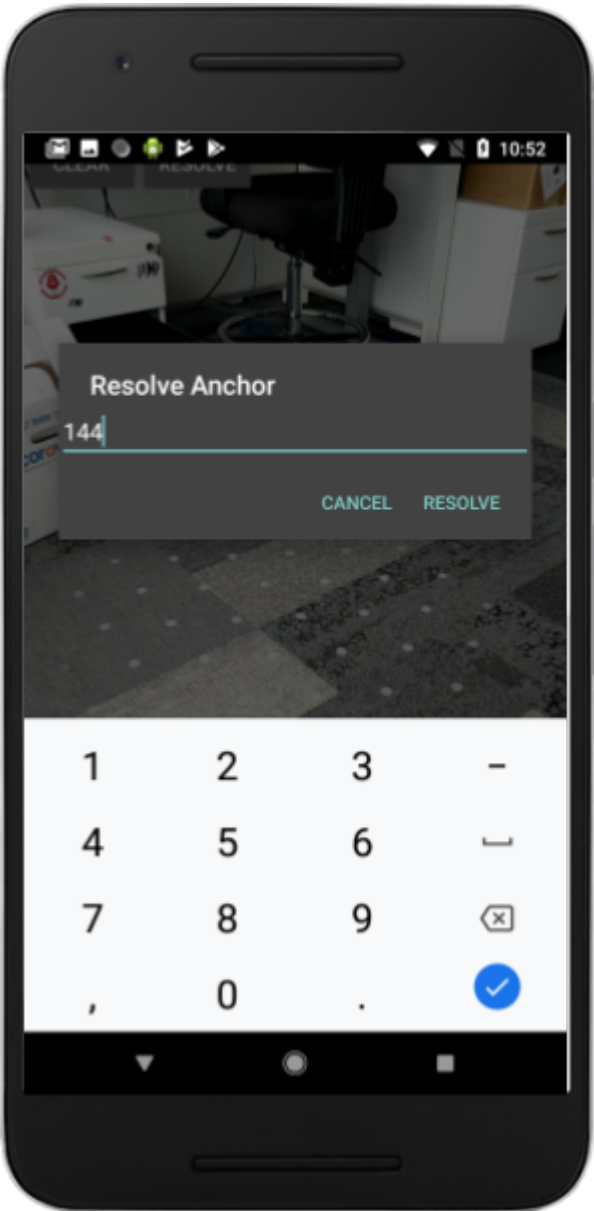
private void onResolveButtonPressed() {
    ResolveDialogFragment dialog = ResolveDialogFragment.createWithOkListener(
        this::onShortCodeEntered);
    dialog.show(getActivity().getSupportFragmentManager(), "Resolve");
}

```

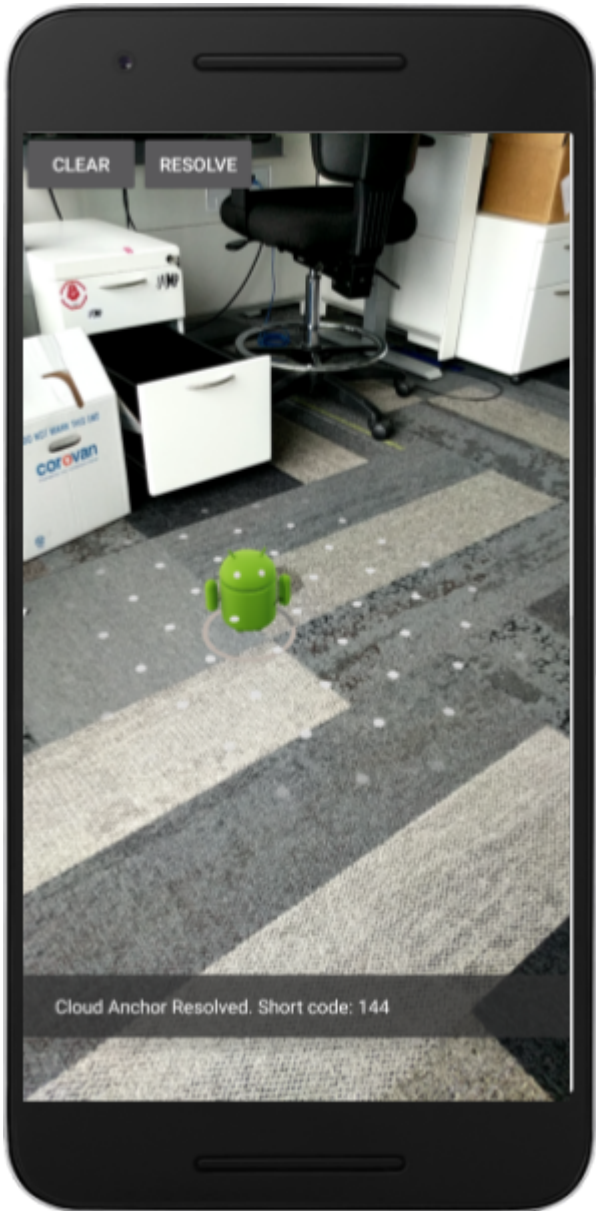
從 Android Studio 建構及執行應用程式，然後執行下列步驟：

1. 在平面上建立錨點，並等待錨點代管完成。
記住簡短代碼。
2. 按下「清除」**CLEAR**按鈕即可刪除錨點。
3. 按下「解決」按鈕。輸入步驟 1 中的短碼。
4. 您應該會看到錨點位於環境中的相同位置。
5. 結束並終止應用程式，然後重新開啟。
6. 重複步驟 (3) 和 (4)。您應該會看到新的錨點，位置與先前相同。

輸入短碼



錨點已順利解析



5. 在裝置之間分享

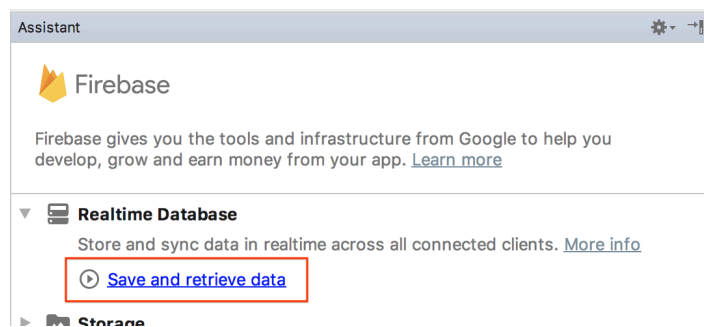
您已瞭解如何將錨點的 Cloud Anchor ID 儲存至裝置的本機儲存空間，並在稍後擷取該 ID 以重建相同的錨點。不過，只有在不同裝置之間共用 Cloud Anchor ID 時，才能完全發揮 Cloud Anchors 的潛力。

應用程式分享 Cloud Anchor ID 的方式由您決定。您可以使用任何方式將字串從一部裝置轉移到另一部裝置。在本程式碼研究室中，您將使用 [Firebase 即時資料庫](#)，在應用程式執行個體之間轉移 Cloud Anchor ID。

設定 Firebase

您必須使用 Google 帳戶設定 Firebase 即時資料庫，才能搭配這個應用程式使用。Android Studio 中的 Firebase Assistant 可輕鬆完成這項作業。

在 Android Studio 中，依序點選「Tools」>「Firebase」。在彈出的「助理」窗格中，按一下「Realtime Database」，然後按一下「儲存及擷取資料」：

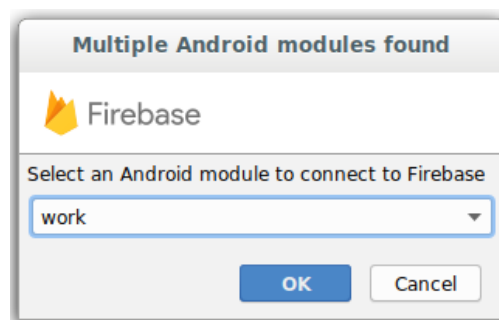


按一下「Connect to Firebase」按鈕，將 Android Studio 專案連結至新的或現有的 Firebase 專案。

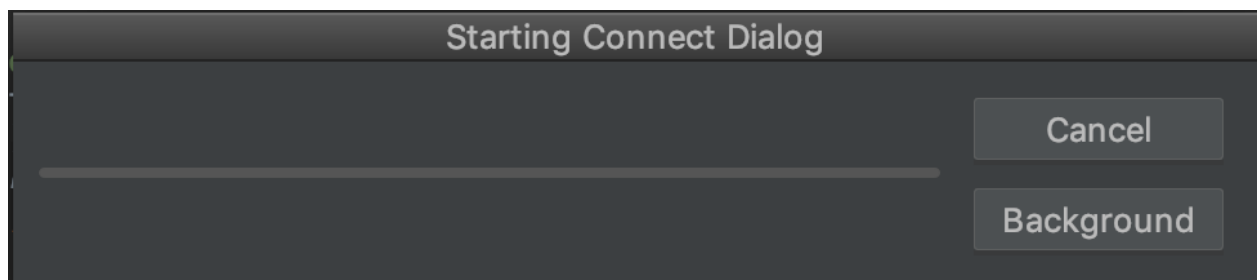
1 Connect your app to Firebase

Connect to Firebase

系統會提示你選取模組。選取 `work` 模組：

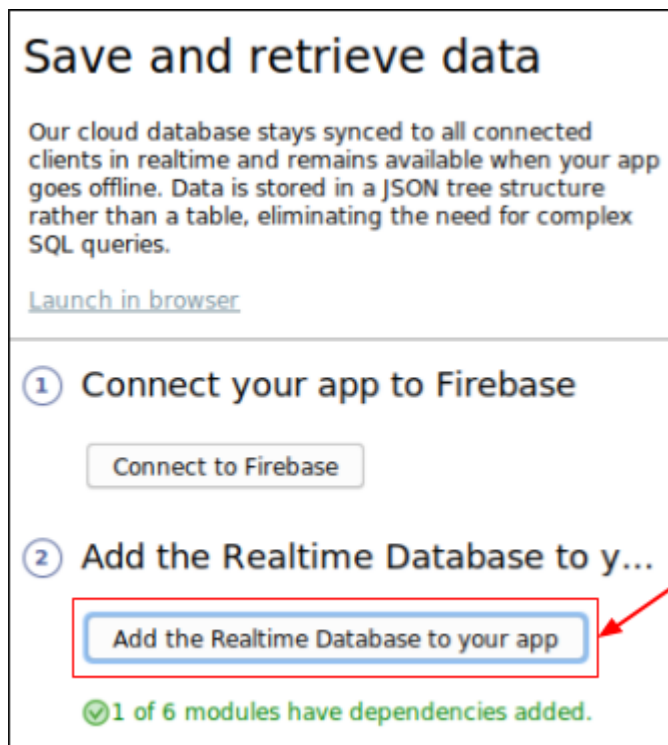


系統會顯示「開始連線」對話方塊。這項作業可能需要一些時間才能完成。

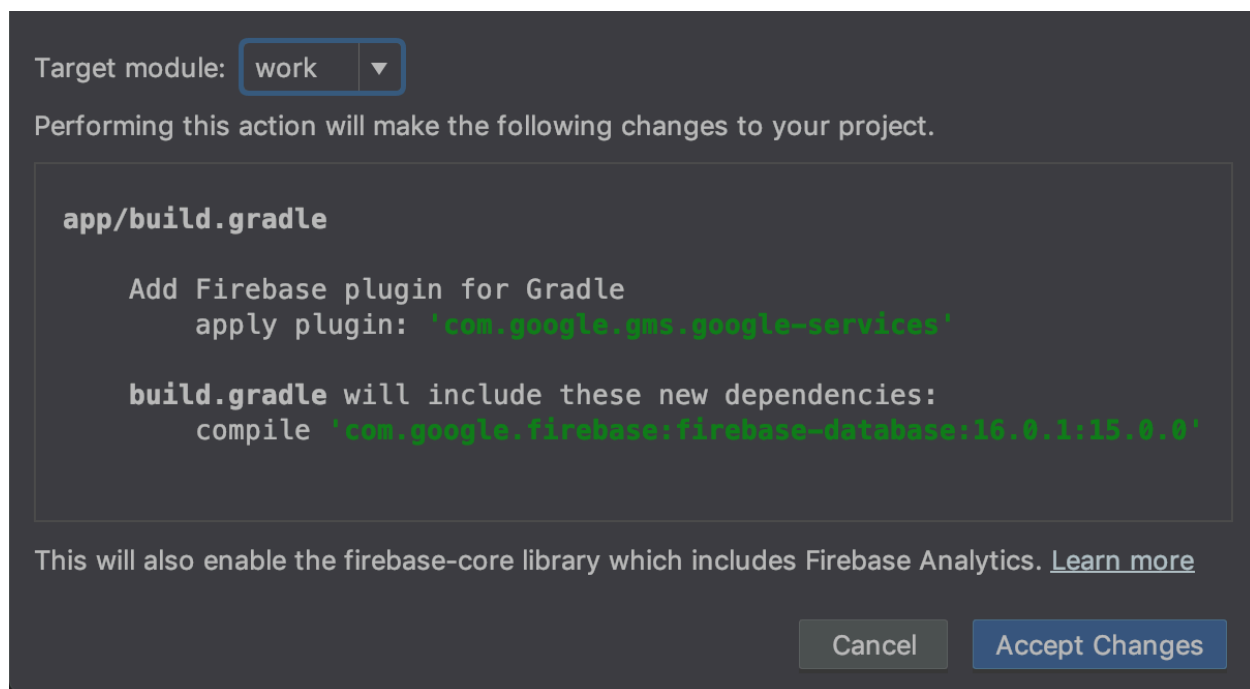


使用 Google 帳戶登入，然後完成網頁工作流程，為應用程式建立 Firebase 專案，直到返回 Android Studio 為止。

接著，在「Assistant」窗格中，按一下「add the Realtime Database to your app」(將 Realtime Database 新增至應用程式)：



在彈出的對話方塊中，從「目標模組」下拉式選單選取「work」，然後按一下「接受變更」。



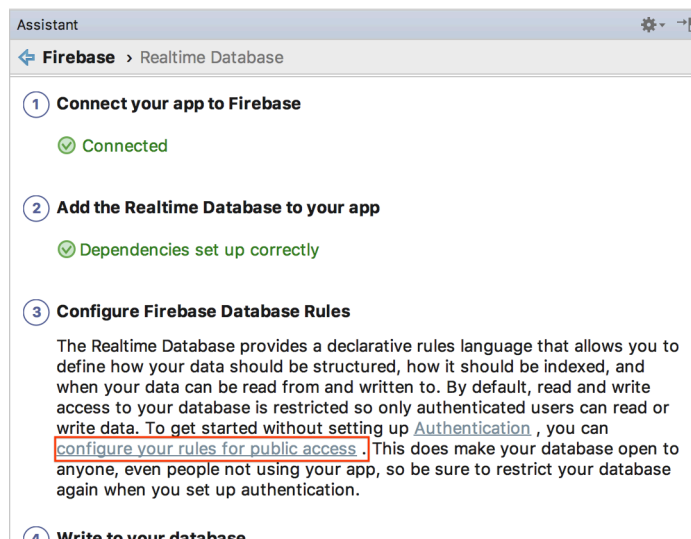
系統接著就會：

1. 在 `work` 目錄中新增 `google-services.json` 檔案
2. 在同一個目錄中，將幾行程式碼新增至 `build.gradle` 檔案。
3. 建構並執行應用程式 (您可能會看到有關 Firebase 資料庫版本號碼的解決錯誤)。

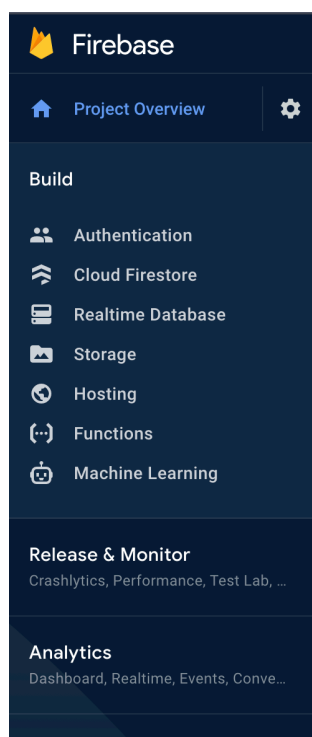
在 `work` 模組 `build.gradle` 檔案中，找出並移除下列程式碼行 (`xxxx` 是最新版本號碼的預留位置)

```
dependencies {  
    ...  
    implementation 'com.google.firebase:firebase-database:xxx'
```

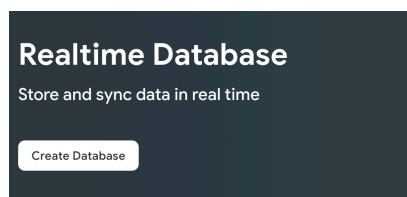
接著，請先查看「[設定公開存取規則](#)」頁面中連結的操作說明，瞭解如何將 Firebase 即時資料庫設為可供全球寫入，但暫時不要按照說明操作。這有助於簡化本程式碼研究室的測試：



在 [Firebase 控制台](#) 中，選取您連結 Android Studio 專案的專案，然後選取「BUILD」>「Realtime Database」。

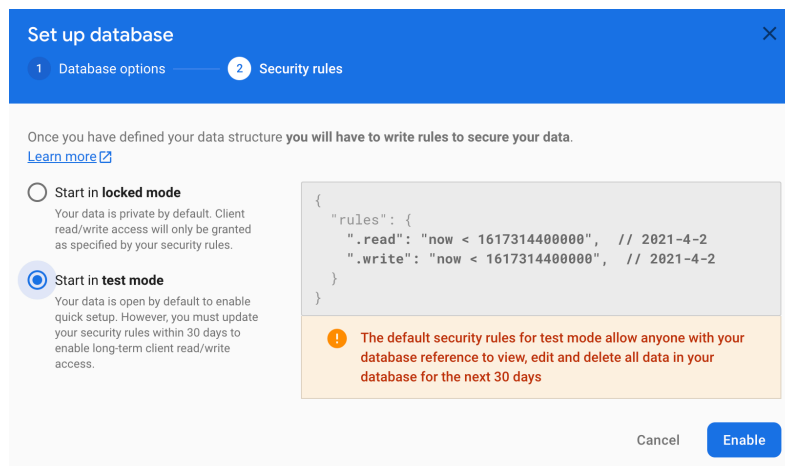


按一下「建立資料庫」，設定 **Realtime Database**：



選擇任何資料庫位置。

在下一個步驟中，選取**測試模式**安全性規則，然後按一下「啟用」：



Set up database [X]

1 Database options — 2 Security rules

Once you have defined your data structure you will have to write rules to secure your data.
[Learn more](#)

☐ Start in **locked mode**
Your data is private by default. Client read/write access will only be granted as specified by your security rules.

☒ Start in **test mode**
Your data is open by default to enable quick setup. However, you must update your security rules within 30 days to enable long-term client read/write access.

```
{
  "rules": {
    ".read": "now < 1617314400000", // 2021-4-2
    ".write": "now < 1617314400000", // 2021-4-2
  }
}
```

! The default security rules for test mode allow anyone with your database reference to view, edit and delete all data in your database for the next 30 days

Cancel **Enable**

您的應用程式現在已設定為使用 Firebase 資料庫。

使用 **FirebaseManager**

現在請將 `StorageManager` 換成 `FirebaseManager`。

在 Android Studio 中，找出 `work` 目錄下的 `CloudAnchorFragment` 類別。將 `StorageManager` 替換為 `FirebaseManager`：

```
// Find this line.
private final StorageManager storageManager = new StorageManager();

// And replace it with this line.
private FirebaseManager firebaseManager;
```

在 `onAttach` 方法中初始化 `firebaseManager`：

```
public void onAttach(@NonNull Context context) {
    super.onAttach(context);
    tapHelper = new TapHelper(context);
    trackingStateHelper = new TrackingStateHelper(requireActivity());

    // The next line is the new addition.
    firebaseManager = new FirebaseManager(context);
}
```

按照下列方式修改 `onShortCodeEntered` 方法：


```
private void onShortCodeEntered(int shortCode) {
    firebaseManager.getCloudAnchorId(shortCode, cloudAnchorId -> {
        if (cloudAnchorId == null || cloudAnchorId.isEmpty()) {
            messageSnackbarHelper.showMessage(
                getActivity(),
                "A Cloud Anchor ID for the short code " + shortCode + " was not found.");
            return;
        }
        resolveButton.setEnabled(false);
        future = session.resolveCloudAnchorAsync(
            cloudAnchorId, (anchor, cloudState) -> onResolveComplete(anchor, cloudState,
            shortCode));
    });
}
```

然後，按照下列方式修改 `onHostComplete` 方法：

```
private void onHostComplete(String cloudAnchorId, CloudAnchorState cloudState) {
    if (cloudState == CloudAnchorState.SUCCESS) {
        firebaseManager.nextShortCode(shortCode -> {
            if (shortCode != null) {
                firebaseManager.storeUsingShortCode(shortCode, cloudAnchorId);
                messageSnackbarHelper.showMessage(getActivity(), "Cloud Anchor Hosted. Short code: "
+ shortCode);
            } else {
                // Firebase could not provide a short code.
                messageSnackbarHelper.showMessage(getActivity(), "Cloud Anchor Hosted, but could not
"
                + "get a short code from Firebase.");
            }
        });
    } else {
        messageSnackbarHelper.showMessage(getActivity(), "Error while hosting: " +
cloudState.toString());
    }
}
```

建構及執行應用程式。您應該會看到與上一節相同的 UI 流程，但現在系統會使用線上 Firebase 資料庫儲存 Cloud Anchor ID 和短代碼，而非裝置本機儲存空間。

多使用者測試

若要測試多使用者體驗，請使用兩部不同的手機：

1. 在兩部裝置上安裝應用程式。
2. 使用一部裝置代管錨點並產生短碼。
3. 使用其他裝置，透過該短代碼解決錨點問題。

您應該可以從一部裝置代管錨點、取得短代碼，並在另一部裝置上使用短代碼，在相同位置查看錨點！

步驟 6 · 約 0 分鐘

6. 總結

恭喜！您已完成本程式碼研究室！

涵蓋內容

- 瞭解如何使用 ARCore SDK 代管錨點，並取得 Cloud Anchor ID。
- 如何使用 Cloud Anchor ID 解析錨點。
- 如何在同一部裝置或不同裝置上，在不同 AR 階段之間儲存及共用 Cloud Anchor ID。

瞭解詳情

- 請參閱 [Android 適用的 Cloud Anchors 總覽](#)
-